

How I Completed Exercise 01

Using the Windows Subsystem for Linux

Preparing WSL

Before starting this, I installed Windows Subsystem for Linux (WSL) on a Windows 10 machine. I found the instructions for this from Microsoft, and I chose the Ubuntu Linux image from the Microsoft store. I would have preferred to try out Centos or Fedora, since I think these would be a better approximation of the common platform environment. However, the Microsoft store wanted to charge me (a small amount) for these. I wasn't sure how well this was going to work the first time I tried it, so I went with one of the free options.

After starting an instance of my WSL Ubuntu, I had to install gcc inside it. Trying to run gcc gives you a message that it's not installed, but it also tells you what to do to install it. I also had to run "sudo apt-get update" before gcc would install, but the Ubuntu image also gives instructions about how to do this.

From the WSL shell prompt, it looks like you can get to your Windows file systems. So, my plan is to store files under my Windows home directory and access them from WSL whenever I need to use some Linux tool (like a compiler or debugger). Each Windows drive shows up as a driver letter under the /mnt directory. I made a symbolic link to my Windows home directory from my home directory in WSL so I could easily change directory (cd) to this directory to get to my home directory files from my WSL environment. For me, this took the following commands. Depending on where your home directory is, you will have to change these a little to get them to work in your environment (e.g., your windows username probably isn't sturgill).

```
$ ln -s /mnt/c/Users/sturgill windows-home
$ cd windows-home
```

Starting the Exercise

Inside my windows home directory, I made a subdirectory to work on CSC 230 assignments, with a directory under it for exercises and a directory under that for the first exercise.

```
$ mkdir csc230
$ cd csc230
$ mkdir ex
$ cd ex
$ mkdir ex01
$ cd ex01
```

Inside the ex01 directory, I copied down the starter files. I used the **scp** command given in the exercise 1 write-up, but I could also have just downloaded the files from the course webpage and saved them under the **230/ex/ex01** directory I made above. You'll need to use your actual *unity-id* in this command. To paste text into the WSL terminal, there's a menu option you can get to from the icon in the upper left corner of the WSL window.

```
$ scp unity-id@remote.eos.ncsu.edu:/mnt/coe/workspace/csc/CSC230/ex/exercise01/* .
```

Don't miss this dot
at the end.

Completing the Implementation

To edit the source file, I tried a couple of different things. You only need one of them, but I wanted to try out some different options. Since you can store your source files in your Windows home directory, you can, of course, open these files from the file browser, using any editor you want. You just need to navigate to the directory you're working in and open a file using the editor of your choice. Also, it looks like the Microsoft WSL installation configures your Linux image so you can start some Windows programs from the WSL shell prompt. From inside the WSL window, I was able to start notepad with the following command. The & at the end says to run notepad in background, so I can still enter more commands from the shell while I'm running the editor.

```
$ notepad.exe arithmetic.c &
```

This worked fine, but you'd probably want to install a more sophisticated programmer's editor if you plan to edit files this way. Also, I tried just using the vim editor from inside the WSL window. This worked fine, but it requires you to be comfortable getting around in vim. If you do run vim like the following, it will run right in your WSL window. So, you'll have to exit vim each time you want to compile your program (or, maybe just suspend it).

```
$ vim arithmetic.c
```

Building and Testing

After adding the needed code to the program using a text editor, I compiled the program from the WSL terminal window:

```
$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
```

If my program hadn't compiled cleanly, I'd have gone back to the editor and fixed whatever was wrong. Since it compiled OK, I'm ready to try it out. In shell code below, I run the program, sending its output to a file named **output.txt**. Right after the program finishes, I check its exit status to make sure it reported that it ran successfully. Finally, I use the **diff** program to make sure my output (saved to the output.txt file) looks exactly like the expected output:

```
$ ./arithmetic > output.txt
$ echo $?
0
$ diff output.txt expected.txt
```

Both of these tests look good, so now I have a choice. I can turn it in as is and hope it will work the same on a common platform machine, or I can try it out on a real common platform machine first, just to be sure.

Checking on the Common Platform

Let's take the safe path.

From my WSL system, I used `sftp` to upload the source file and expected output to AFS. The `sftp` program is a command-line tool that lets you upload or download files from a remote system. It uses the same protocol as the `scp` program you may have used when you copied the starter files at the start of the exercise. If you're accustomed to using FileZilla, `sftp` does a similar job, but, instead of giving you a graphical interface, it lets you copy files back and forth by typing commands at a prompt. If you stored your source files under your Windows home directory like I did, then you could also use any Windows tool you want to upload a copy to NCSU Drive.

I used the following commands to copy my work up to one of the remote Linux machines. You should be able to use similar commands, but you'll need to use your own unity ID in the `sftp` command.

```
$ sftp unity-id@remote.eos.ncsu.edu
Connecting to remote.eos.ncsu.edu...
unity-id@remote.eos.ncsu.edu's password:
sftp> mkdir csc230
sftp> cd csc230
sftp> mkdir ex
sftp> cd ex
sftp> mkdir ex01
sftp> cd ex01
sftp> put arithmetic.c
Uploading arithmetic.c to /mnt/ncsudrive/.../ex01/arithmetic.c
sftp> put expected.txt
Uploading expected.txt to /mnt/ncsudrive/.../ex01/expected.txt
sftp> exit
```

Then, I ran `ssh` inside my WSL image to log in to an EOS linux machine to try out my program. You could use any ssh client you want to do this (e.g., putty or MobaXterm). Be sure to replace the `unity-id` below with your actual unity ID. After authenticating with my NCSU credentials, I change directory to the place where I uploaded my exercise 1 files. I compile the program and then run it. Finally, I check the exit status and compare the program's output against the expected output.

```
$ ssh unity-id@remote.eos.ncsu.edu
unity-id@remote.eos.ncsu.edu's password:
Last login: ... from ...
eos$ cd csc230/ex/ex01
eos$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
eos$ ./arithmetic > output.txt
eos$ echo $?
0
eos$ diff output.txt expected.txt
eos$ exit
```

Good, it worked just as well on the EOS Linux machine as it did on my WSL image.

Submitting your Solution

I'm ready to turn it in. Since the source file I wrote is really stored under my Windows home directory, I can submit from the Windows side. I just need to start up a web browser in Windows and visit the Moodle page for CSC 230. Clicking on the assignment for exercise 01 took me to a submission page where I chose "Add Submission". From there, you can drag a copy of your solution into the submission box. Select your **arithmetic.c** file for submission then choose "Save changes" down near the bottom of the web page.